Contents

Basic Linux Commands 4
Navigation (ls, cd, pwd) 4
File Operations (cp, mv, rm, nano) 4
Disk Usage (df, du) 4
Process Monitoring (ps, top, htop) 5
Users & Permissions 5
File Permissions (chmod, chown) 6
Server Access and Security 7
SSH Basics 7
Firewall Setup (UFW) 8
Cockpit 8
Updating & Patching 9
Service Management 11
Collaborative Shared Folder 12
Web Server Setup 12
Apache vs. Nginx: An Overview 12
Installing Apache (The LAMP Stack) 13
Installation 13
Virtual Hosts (Hosting Multiple Sites) 13
Installing Nginx (The LEMP Stack) 14
Installation 14
Server Blocks (Nginx's "Virtual Hosts") 14
Installing PHP & Extensions 15
PHP-FPM Setup 15
Setting up PHP-FPM for Nginx 16
Setting up PHP-FPM for Apache 16
Database Management (MySQL/MariaDB) 17
Install the Server 17
Secure the Installation 17

Creating Users & Databases 17
Deploying a Sample App 18
Uploading Code 18
The Git Workflow: 18
Setting Permissions (www-data) 19
Configuring .env (Database Connection) 19
Application Services 20
SMTP & Email Setup 20
Configure Postfix (Local Mail Only) 20
Connecting to External SMTP (The Standard) 20
SSL/TLS Certificates (Let's Encrypt) 21
Auto-Renewal 21
Cron Jobs & Scheduling 22
Background Jobs & Queues (Supervisor) 23
Security & Optimization 24
SSH Hardening 24
Fail2Ban (The Automated Bouncer) 24
File & Directory Hardening 25
Caching Overview 25
Configuring PHP OPCache 26
Installing Redis (Object Cache) 26
PHP-FPM Tuning 26
Database Optimization Basics 27
MySQLTuner 27
Creating an Index 27
Monitoring & Maintenance 28
Log Management (/var/log/) 28
Resource Monitoring 28
Automating Security Updates 29
Backups 29

File System Backups 29
A. Archiving (tar) 29
Mirroring (rsync) 30
Remote Backup Strategies 30

Basic Linux Commands

Navigation (ls, cd, pwd)

Command	Description
pwd	Print Working Directory. Shows the full path of where you are currently located.
ls	List. Shows the files and folders in your current directory.
ls -l	Lists files in "Long" format, showing permissions, owner, size, and date modified.
ls -a	Lists All files, including hidden files (files starting with a . like .bashrc).
ls -la	Combines options: Lists all files in long format.
ls -lh	Long list with Human-readable file sizes (e.g., shows 5MB instead of 5242880).
cd [folder]	Change Directory. Moves you into the specified folder name.
cd ..	Moves you Up one level to the parent directory.
cd ../..	Moves you Up two levels at once.
cd ~ (or just cd)	returns you immediately to your user's Home directory (e.g., /home/username).
cd -	Switches you back to the Previous directory you were in (like a "Back" button).
cd /	Moves you to the Root of the file system (the very top level).

File Operations (cp, mv, rm, nano)

Command	Description
touch [file]	Creates an empty file (or updates the timestamp if it already exists).
mkdir [folder]	Make Directory. Creates a new folder.
mkdir -p a/b/c	Creates a nested directory path (parent and children) all at once.
cp [source] [dest]	Copy a file from source to destination.
cp -r [folder] [dest]	Recursive copy. Required to copy a folder and everything inside it.
cp -i [file] [dest]	Interactive. Asks for confirmation before overwriting an existing file.
mv [source] [dest]	Move. Moves a file or folder from one place to another.
mv [old_name] [new_name]	"Rename. If the destination is just a new name in the same folder, Linux renames the file."
rm [file]	Remove. Deletes a file permanently (Does not go to a Recycle Bin).
rm -r [folder]	Recursive remove. Required to delete a folder and its contents.
rm -rf [folder]	Recursive Force. Deletes a folder immediately without prompting. Dangerous!

Disk Usage (df, du)

Command	Description
df	Disk Free. Shows total, used, and available space for entire file systems (partitions).
df -h	Human-readable. Displays sizes in GB and MB instead of KB blocks. The standard check.
du	Disk Usage. Estimates file space usage for the current directory and subdirectories.
du -h	Displays the size of files and folders in a Human-readable format.
du -sh [folder]	Summary Human-readable. Shows the *total* size of a specific folder (without listing every single file inside it).

du -h --max-depth=1	Shows the size of immediate subfolders only—perfect for hunting down which folder is hogging space.

Process Monitoring (ps, top, htop)

Command	Description
top	Displays a real-time, dynamic view of running processes. Shows CPU and RAM usage. (Press q to quit).
htop	A friendlier, colorful, interactive version of top. Allows scrolling and mouse clicks. (Often needs installing: sudo apt install htop).
ps	Process Status. Shows a static snapshot of processes running in the *current* terminal.
ps aux	The "Show Me Everything" command. Lists All processes from all Users, including those without a terminal (x).
ps aux | grep [name]	Searches the process list for a specific program name (e.g., `ps aux
kill [PID]	Forces a process to stop using its Process ID (PID).
kill -9 [PID]	"Force Kill." Instantly terminates the process without allowing it to clean up. Use as a last resort.

Users & Permissions

Command	Description
sudo adduser [name]	Creates a new user. Recommended for Ubuntu/Debian as it automatically creates a Home folder and asks for a password.
sudo useradd [name]	The "raw" command. Creates a user *without* a home folder or password prompt. Avoid teaching this to beginners unless necessary for scripting.
sudo deluser [name]	Deletes a user account but leaves their files behind.
sudo deluser --remove-home [name]	Deletes the user and their home directory.
sudo addgroup [group]	Creates a new group.
sudo usermod -aG [group] [user]	Append to Group. Adds a user to a group without removing them from their existing groups.
sudo usermod -aG sudo [user]	Grants a user Administrator (Root) privileges.

Command	Description
cat /etc/passwd	View All Users. Lists every account on the system. (Tip: Use cut -d: -f1 /etc/passwd to see just the usernames).
cat /etc/group	View All Groups. Lists every group and the Group ID (GID).
getent group [name]	View Group Members. Shows who is actively in a specific group (e.g., getent group sudo).
sudo usermod [options] [user]	Modify User. Used to change account properties.
sudo usermod -l [new] [old]	Renames a user account (User must be logged out).

sudo usermod -s /bin/bash [user]	Changes the user's default shell (e.g., from sh to bash).
sudo usermod -L [user]	Locks the account (user cannot log in).
sudo usermod -U [user]	Unlocks the account.
sudo groupmod -n [new] [old]	Modify Group. Renames a group.
sudo chage [options] [user]	Password Policies. Manages password expiration.
sudo chage -l [user]	List. View the current expiration settings for a user.
sudo chage -d 0 [user]	Force Change. Forces the user to change their password on the very next login.
sudo chage -M 90 [user]	Sets the password to expire every 90 days.

Important User Accounts & Groups

- Root (UID 0): The superuser. Has unrestricted access to everything. In Ubuntu, this account is usually "locked" by default (no password set), forcing you to use sudo.
- System Accounts (UID < 1000): You will see users like www-data (web servers), sys, or nobody. These are non-human accounts used by background services to limit security risks. If a hacker compromises the web server, they only get www-data permissions, not Root.
- Nobody: An account specifically designed to own nothing and have no privileges. Used for running untrusted processes.

The wheel Group

- In the Unix world (like RedHat or BSD), the wheel group is the designated administrative group. Only members of wheel can use the su command to become root.

- Ubuntu does not use the wheel group by default (though you can enable it). Instead, Ubuntu uses the sudo group. If you see a reference to wheel in an online tutorial, translate that to sudo for Ubuntu.

Sudoers

- The file /etc/sudoers dictates who can use sudo and what they can do.
- The Golden Rule: NEVER edit this file with a standard text editor. If you make a typo, you can lock yourself out of the system permanently.
- Always use sudo visudo. This opens the file safely and checks for syntax errors before saving.
- username host=(ALL:ALL) ALL (Gives a user full root access).

File Permissions (chmod, chown)

Command / Variation	Description
ls -l	The prerequisite command. You must run this to *see* the current permissions (e.g., -rwxr-xr-x).
chown [user]:[group] [file]	Change Owner. Changes who owns the file.
chown -R [user]:[group] [folder]	Recursive. Changes ownership of a folder and everything inside it.
chmod [mode] [file]	Change Mode. Modifies read/write/execute permissions.

chmod +x [script.sh]	Symbolic Mode. Adds (+) Execute permission for everyone. Easy to remember.
chmod 755 [file]	Numeric Mode. Owner gets full access (7), Group and Others get Read/Execute (5).
chmod 600 [file]	Private Mode. Owner gets Read/Write (6), everyone else gets nothing (0). Standard for SSH keys.

Access Control Lists (ACL)
Standard permissions are limited (only one owner, only one group). What if Bob owns the file, the Developers group has read access, but you specifically want Alice (who isn't in the group) to have Write access? This is where ACLs come in.

You may need to install the tools first:

```
sudo apt install acl
```

Key Commands:

getfacl [file]	View the current specific ACLs.
setfacl	Set specific user permissions.

Examples:
Give specific user 'alice' Read/Write access to a file owned by someone else

```
setfacl -m u:alice:rw project_file.txt
```

Remove permissions for a specific user

```
setfacl -x u:alice project_file.txt
```

Remove ALL special ACL entries (reset to default)

```
setfacl -b project_file.txt
```

Server Access and Security

SSH Basics

Command / Variation	Description
ssh [user]@[host]	Connects to a remote server (e.g., ssh student@192.168.1.50).
ssh -p [port] [user]@[host]	Connects using a custom port (if the server isn't listening on the default port 22).
ssh -i [keyfile] [user]@[host]	Connects using a specific Identity (private key) file instead of the default.
ssh-keygen -t rsa -b 4096	Generates a new Public/Private key pair on your local machine.
ssh-copy-id [user]@[host]	Copies your Public key to the remote server, enabling password-less login.
sudo systemctl restart ssh	Restarts the SSH service (required after changing /etc/ssh/sshd_config).

Firewall Setup (UFW)

Uncomplicated Firewall (UFW) is the default Ubuntu firewall tool. It is "Deny all incoming" by default, so you must explicitly allow services.

The "Lockout" Danger
Allow SSH First: Always run ***sudo ufw allow ssh*** before enabling the firewall.

Command / Variation	Description
sudo ufw status	Checks if the firewall is active or inactive.
sudo ufw status verbose	Shows detailed rules and default policies (Default is usually "Deny Incoming").
sudo ufw enable	Turns the firewall ON. (Warning: Ensure SSH is allowed first!).
sudo ufw disable	Turns the firewall OFF.
sudo ufw allow [service]	Opens a port by name (e.g., sudo ufw allow ssh, sudo ufw allow http).
sudo ufw allow [port]/[proto]	Opens a specific port number (e.g., sudo ufw allow 8080/tcp).
sudo ufw allow from [IP]	Whitelisting. Allows connections *only* from a specific IP address (e.g., your office VPN).
sudo ufw deny [port]	Explicitly blocks traffic on a port.
sudo ufw delete allow [port]	Removes a rule (e.g., stops allowing port 80).

Cockpit

Cockpit is a "Server Administration Dashboard" that runs in a web browser. It is the perfect bridge for students who are transitioning from Windows/GUI environments to Linux Command Line.
Instead of typing commands to check disk space or restart a service, they can click buttons, while still having access to a built-in terminal.

Install the Package

```
sudo apt update
sudo apt install cockpit -y
```

Start the Socket Cockpit uses a "Socket" rather than a constantly running heavy service. It only activates when you connect to it.

```
sudo systemctl enable --now cockpit.socket
```

Verify Status

```
systemctl status cockpit.socket
```

Output should say: Active: active (listening)

Accessing the Web Interface
https://<SERVER_IP>:9090

The "Not Secure" Warning:

- Your browser will block the page saying "Your connection is not private."
- The Reason: The server is using a "Self-Signed Certificate." It is encrypted, but no public authority (like Google or Verisign) has vouched for it.
- The Fix: Click Advanced -> Proceed to [IP] (unsafe).

Login: Use your standard Linux username and password (the same ones you use for SSH).

Updating & Patching

Updating lists (update) refreshes the catalog of available software versions. Upgrading (upgrade) actually installs the new versions.

Command / Variation	Description
sudo apt update	Refreshes the List. Checks the repositories to see what new versions are available. Does not install anything.
sudo apt list --upgradable	View which packages actually have updates waiting after running update.
sudo apt upgrade	Installs Updates. Upgrades all current packages to the newest version found in the list.
sudo apt upgrade -y	Answers "Yes" automatically to the "Do you want to continue?" prompt.
sudo apt dist-upgrade	Smart Upgrade. Handles changing dependencies (e.g., if a new update requires removing an old package to work, this command handles it).
sudo apt autoremove	Cleanup. Removes "orphan" packages that were installed as dependencies but are no longer needed.

Action	New Command (Use this)	Old Command (Legacy)
Install	apt install firefox	apt-get install firefox
Remove	apt remove firefox	apt-get remove firefox
Update List	apt update	apt-get update
Upgrade All	apt upgrade	apt-get upgrade
Search	apt search firefox	apt-cache search firefox
Show Details	apt show firefox	apt-cache show firefox
Full Upgrade	apt full-upgrade	apt-get dist-upgrade

To control specifically who can manage packages (install/update software) without giving them full root access to the rest of the system, you must edit the Sudoers configuration.

Scenario: Creating a "Software Manager" Role
We want to allow the user junior to run apt and dpkg commands, but nothing else.

1. Define the Command AliasIt is best practice to create a "Group" of commands (Alias) rather than listing them one by one for every user. Open the editor

```
sudo visudo
```

Add the following lines at the bottom of the file (or ideally, create a new file in /etc/sudoers.d/software_managers):

```
# 1. Create a name for the group of commands
Cmnd_Alias SOFTWARE = /usr/bin/apt, /usr/bin/apt-get, /usr/bin/dpkg, /usr/bin/snap

# 2. Grant the user 'junior' access to this group
junior ALL=(ALL:ALL) SOFTWARE
```

2. Breakdown of the Syntax

Part	Description
Cmnd_Alias	Defines a list of commands under one name (SOFTWARE).
/usr/bin/apt	Full Path Required. You must list the exact location of the command for security.
Junior	The username you are restricting.
ALL=(ALL:ALL)	Host=(TargetUser). "On all hosts, as any user."
SOFTWARE	The specific command alias they are allowed to run.

Example (Prove they are restricted):
Log in as junior and try to reboot the server.

```
su - junior
sudo reboot
```

Output: Sorry, user junior is not allowed to execute '/usr/sbin/reboot' as root.

Prove they can manage packages:
Now try to update the system.

```
sudo apt update
```

Output: (Works successfully)

The "Shell Escape" Risk
Privilege Escalation - If you give a user the ability to install software, you have effectively given them the ability to become root, even if you tried to restrict them.

The Hack: A smart user could build a custom .deb package. Inside that package is a script that says "Make me a root user."

The Result: When they run sudo dpkg -i malicious_package.deb, the system executes that script as root.

Only grant package management permissions to users you trust completely. It is almost equivalent to giving them full root access.

User Role	Sudoers Configuration	Capabilities
Standard User	(Not in sudoers)	Can run software, but cannot install/remove anything.
Package Manager	user ALL=(root) /usr/bin/apt	Can install/remove software. Cannot reboot, change passwords, or edit system config.
Full Admin	user ALL=(ALL:ALL) ALL	Can do everything (members of sudo group).

Service Management

Command	Description
systemctl status [service]	Checks if a service is running, failed, or dead. Shows the last few log lines.
systemctl start [service]	Immediately starts the service in the current session.
systemctl stop [service]	Immediately stops the service.
systemctl restart [service]	Stops and then Starts the service again.
systemctl reload [service]	Reloads the configuration files *without* killing the process.
systemctl enable [service]	Configures the service to start automatically at boot time.
systemctl disable [service]	Prevents the service from starting at boot.
systemctl is-active [service]	Returns a simple "active" or "inactive" (good for scripts).
systemctl list-units --type=service	Lists all currently running services on the entire system.

Troubleshooting Services (journalctl)
When a service fails (e.g., systemctl start gives an error), it rarely tells you why on the screen. You must check the logs.

Systemd uses a centralized logging system called the Journal.

Command	Description
journalctl -u [service]	Shows logs specifically for one Unit (service).
journalctl -xe	Jump to the End of the logs with eXtra details. The standard "Oh no, something broke" command.
journalctl -f	Follow. Shows live logs as they happen (scrolling). Great for watching while you try to reproduce an error.
journalctl --since "1 hour ago"	Filters logs to show only recent events.
journalctl -p err	Shows only messages with priority Error or worse (filters out "info" noise).

Collaborative Shared Folder

You have a project folder (/srv/project). You want Andy and Ben to both be able to read, write, and delete files inside this folder, regardless of who created them.

By default in Linux, when Andy creates a file, it is owned by andy:andy. Even if Ben has access to the folder, he cannot edit Andy's file because he belongs to the group ben, not andy.

Set Group ID (SGID)
We need to force every file created in this folder to automatically inherit a shared group.

1. Create the Shared Group Create a group for the collaboration.

```
sudo addgroup developers
```

2. Add Users to the Group Add Andy and Ben to the group.

```
sudo usermod -aG developers andy
sudo usermod -aG developers ben
```

(Note: They must log out and log back in for this to take effect!)

3. Create the Directory and Set Group Ownership

```
sudo mkdir /srv/project
sudo chown :developers /srv/project
```

Current Status: The folder is owned by Root, but the Group is 'developers'.

4. We set the Set Group ID bit on the directory.

```
sudo chmod g+s /srv/project
```

Any file created inside this folder will inherit the Group of the folder (developers), NOT the primary group of the user who created it.

5. Ensure Group Write Access Finally, make sure the group actually has write permissions.

```
sudo chmod 775 /srv/project
```

Web Server Setup

Apache vs. Nginx: An Overview

Feature	Apache (The "Swiss Army Knife")	Nginx (The "Formula 1 Car")
Architecture	Process-based. Creates a new thread/process for every connection. Can be heavy on RAM.	Event-driven. Handles thousands of connections asynchronously. Extremely lightweight.
Configuration	Decentralized. Allows per-directory configuration via .htaccess files. Great for shared hosting.	Centralized. All config happens in the main server files. Faster because it doesn't scan directories for configs.

Best For	Beginners, Dynamic content, shared hosting environments.	High-traffic sites, Static content, Reverse Proxies, Load Balancing.

Installing Apache (The LAMP Stack)

Installation

```
sudo apt update
sudo apt install apache2 -y
sudo systemctl status apache2
```

Open a browser and type http://localhost or the server's IP. You should see the "Apache2 Default Page".

Virtual Hosts (Hosting Multiple Sites)

By default, Apache serves from /var/www/html. To host a specific site (e.g., testsite.local), we create a Virtual Host.

Create the Directory

```
# Create the folder for the site
sudo mkdir -p /var/www/testsite.local/public_html
```

```
# Assign ownership to the current user (for easier editing) and permissions
sudo chown -R $USER:$USER /var/www/testsite.local/public_html
sudo chmod -R 755 /var/www
```

```
# Create a sample index file
echo "<h1>Success! The Apache Virtual Host is working!</h1>" >
/var/www/testsite.local/public_html/index.html
```

Create the Config File

```
# Create a new configuration file in sites-available.
sudo nano /etc/apache2/sites-available/testsite.local.conf
```

Paste this configuration:

```
<VirtualHost *:80>
   ServerAdmin admin@testsite.local
   ServerName testsite.local
   ServerAlias www.testsite.local
   DocumentRoot /var/www/testsite.local/public_html

   ErrorLog ${APACHE_LOG_DIR}/error.log
   CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>
```

Enable the Site

Ubuntu provides helpful shortcuts (a2ensite = Apache2 Enable Site).

```
# Enable the new site
sudo a2ensite testsite.local.conf

# Disable the default site (optional, but good for testing)
sudo a2dissite 000-default.conf

# Check for syntax errors
sudo apache2ctl configtest

# Restart Apache
sudo systemctl restart apache2
```

Installing Nginx (The LEMP Stack)

Note: If you installed Apache above, stop it first (sudo systemctl stop apache2) to avoid port 80 conflicts.

Installation

```
sudo apt update
sudo apt install nginx -y
sudo systemctl status nginx
```

Server Blocks (Nginx's "Virtual Hosts")

Nginx uses "Server Blocks" to achieve the same goal.

Create the Directory (Same as Apache)

```
sudo mkdir -p /var/www/nginx-site.local/html
sudo chown -R $USER:$USER /var/www/nginx-site.local/html
echo "<h1>Success! Nginx Server Block is working!</h1>" > /var/www/nginx-site.local/html/index.html
```

Create the Config File

```
sudo nano /etc/nginx/sites-available/nginx-site.local
```

Paste this configuration:

```
server {
  listen 80;
  listen [::]:80;

  root /var/www/nginx-site.local/html;
  index index.html index.htm index.php;
```

```
    server_name nginx-site.local www.nginx-site.local;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

Enable the Site (Symlink Method)
Nginx does not have a2ensite. You must manually link the file to sites-enabled.

```
# Create the symbolic link
sudo ln -s /etc/nginx/sites-available/nginx-site.local /etc/nginx/sites-enabled/

# Check for syntax errors (Crucial step in Nginx)
sudo nginx -t

# Restart Nginx
sudo systemctl restart nginx
```

Installing PHP & Extensions

Ubuntu 22.04 installs PHP 8.1 by default. Ubuntu 24.04 installs PHP 8.3. The commands below work for both (they grab the default version).

A. Install PHP and Common Extensions

This installs the CLI, the MySQL driver, string handling, image processing, and curl.

```
sudo apt install php-cli php-mysql php-mbstring php-gd php-curl php-xml -y
```

B. Verifying Installation

```
php -v
```

Output should show PHP 8.1 (Ubuntu 22) or 8.3 (Ubuntu 24)

PHP-FPM Setup

PHP-FPM (FastCGI Process Manager) is a high-performance PHP handler.

- ✓ For Nginx: It is required (Nginx cannot process PHP natively).
- ✓ For Apache: It is optional (Apache usually uses libapache2-mod-php by default), but FPM is faster and handles high loads better.

Setting up PHP-FPM for Nginx

Install FPM

```
sudo apt install php-fpm
```

Check the socket version

- Look at /var/run/php/.
- You will see either *php8.1-fpm.sock* or *php8.3-fpm.sock*.

Edit your Nginx Server Block

```
sudo nano /etc/nginx/sites-available/nginx-site.local
```

Add the PHP Location Block
(Replace 8.x with your actual version, e.g., 8.3).

```
location ~ \.php$ {
   include snippets/fastcgi-php.conf;
   fastcgi_pass unix:/var/run/php/php8.3-fpm.sock;
}
```

Restart Nginx

```
sudo systemctl restart nginx
```

Setting up PHP-FPM for Apache

Install FPM

```
sudo apt install php-fpm libapache2-mod-fcgid
```

Enable the modules
Apache needs to activate the proxy modules to talk to FPM.

```
sudo a2enmod proxy_fcgi setenvif
sudo a2enconf php8.3-fpm                    # (Adjust for 8.1 if on Ubuntu 22)
```

Disable the old module
If you were using the standard PHP module, turn it off to avoid conflicts.

```
sudo a2dismod php8.3
```

Restart Apache

```
sudo systemctl restart apache2
```

The "Hosts File" Trick

Q: "I set up testsite.local, but when I type it in Chrome, it searches Google."

A: Since testsite.local isn't a real domain on the internet, the student's computer doesn't know where it is. They must edit their local computer's hosts file to map the IP.

- On their Windows Laptop: Edit C:\Windows\System32\drivers\etc\hosts (run Notepad as Admin).
- On their Linux/Mac Laptop: Edit /etc/hosts.
- Add this line:

```
192.168.x.x  testsite.local  nginx-site.local
```

  (Replace 192.168.x.x with the IP address of their Ubuntu Server).
- Now, the browser will direct that URL straight to their new server.

Database Management (MySQL/MariaDB)

Install the Server

```
sudo apt update
sudo apt install mysql-server -y
```

Secure the Installation

Run the security script included with MySQL. It removes dangerous defaults.

```
sudo mysql_secure_installation
```

- **VALIDATE PASSWORD PLUGIN?** -> Press N (No) for a classroom env (keeps it simple), Y for production.
- **Remove anonymous users?** -> Y
- **Disallow root login remotely?** -> Y
- **Remove test database?** -> Y
- **Reload privilege tables now?** -> Y

Creating Users & Databases

We need to enter the MySQL console. On Ubuntu, the root user uses "auth_socket" by default, meaning you just use sudo to log in (no password needed for root yet).

1. Log in:

```
sudo mysql
```

2. SQL Commands:

```
-- 1. Create the Database
CREATE DATABASE my_app_db;
```

```
-- 2. Create the User
-- REPLACE 'password123' with a secure password!
CREATE USER 'app_user'@'localhost' IDENTIFIED BY 'password123';
```

```
-- 3. Give the Privileges
GRANT ALL PRIVILEGES ON my_app_db.* TO 'app_user'@'localhost';
```

```
-- 4. Refresh the system to apply changes
FLUSH PRIVILEGES;
```

```
-- 5. Exit
EXIT;
```

3. Verifying Access

Test that the *new* user can log in (do not use sudo here):

```
mysql -u app_user -p
# Enter 'password123' when prompted.
# If you see the 'mysql>' prompt, it worked.
```

Deploying a Sample App

Your code currently lives on your laptop (Local). It needs to move to the server (Remote). We generally put web files in /var/www/html or a custom folder like /var/www/my_app.

Uploading Code

Method	Best For...	Command / Tool
Git (Recommended)	Standard workflows. Easy to update later (git pull).	git clone https://github.com/user/repo.git
SCP (Secure Copy)	Quick, one-time transfer of a zip file or single script.	scp -r ./my_code_folder user@server_ip:/tmp
SFTP	Visual learners who prefer a GUI (Drag & Drop).	Use FileZilla or WinSCP.

The Git Workflow:

1. Install Git on Server:

```
sudo apt install git -y
```

2. Clone the Repo:
   Move to the web directory and download the code.

```
cd /var/www
sudo git clone https://github.com/laravel/laravel.git my_app
# (We are using the Laravel framework as a sample, but this applies to any code)
```

Setting Permissions (www-data)

The web server software (Nginx/Apache) runs as a specific user named www-data. If you upload files as root or ubuntu, the web server cannot read or write to them, resulting in 403 Forbidden or 500 Server Error.

The Fix:
Give ownership of the folder to www-data.

```
# 1. Change Owner (User & Group) to www-data
sudo chown -R www-data:www-data /var/www/my_app
```

```
# 2. Fix Directory Permissions (755: Owner writes, everyone reads)
sudo find /var/www/my_app -type d -exec chmod 755 {} \;
```

```
# 3. Fix File Permissions (644: Owner writes, everyone reads)
sudo find /var/www/my_app -type f -exec chmod 644 {} \;
```

"Storage" folders:
Some apps (like WordPress or Laravel) need to write logs or cache. You might need to give write access specifically to those folders:

```
sudo chmod -R 775 /var/www/my_app/storage
```

Configuring .env (Database Connection)

Never hardcode passwords in your PHP/Python files. We use "Environment Variables" stored in a hidden file called .env.

1. Create the file:
   Most apps come with a .env.example. Copy it.

   ```
   cd /var/www/my_app
   sudo cp .env.example .env
   ```

2. Edit the file:

   ```
   sudo nano .env
   ```

3. Update Database Credentials:
   Find the DB section and plug in the details we created in our Database

   ```
   DB_CONNECTION=mysql
   DB_HOST=127.0.0.1
   DB_PORT=3306
   DB_DATABASE=my_app_db
   DB_USERNAME=app_user
   DB_PASSWORD=password123
   ```

4. Secure the .env file:
   This file contains secrets! Ensure only the owner (and web server) can read it.

```
sudo chmod 600 .env
sudo chown www-data:www-data .env
```

Application Services

SMTP & Email Setup

Sending email from a server is difficult because of spam filters.

- **The "Old" Way:** Hosting your own Mail Server (Postfix/Dovecot). Hard to maintain, low deliverability.
- **The "Modern" Way:** Using Postfix as a "Relay" (pass-through) to an external provider like SendGrid, Mailgun, or Amazon SES.
- **The "App" Way:** Configuring the application (Laravel/WordPress) to talk directly to the external provider, bypassing the server's mail system entirely.

Configure Postfix (Local Mail Only)

This is useful for system alerts (e.g., "Cron job failed" or "Disk full").

1. Install Postfix:

```
sudo apt update
sudo apt install mailutils postfix -y
```

2. Configuration Wizard:
   - Type: Select "Internet Site".
   - System Mail Name: Enter your hostname (e.g., server1.example.com).

3. Test it:

```
echo "This is a test email" | mail -s "Test Subject" admin@example.com
```

(Note: This often lands in Spam because it lacks proper DNS records).

Connecting to External SMTP (The Standard)

Instead of relying on Postfix, admins usually configure the application directly.

Configuring Laravel .env

Edit the environment file in your application directory:

```
MAIL_MAILER=smtp
MAIL_HOST=smtp.sendgrid.net     # or smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=apikey            # or your gmail address
MAIL_PASSWORD=your_api_key      # or your app password
```

```
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS="hello@example.com"
```

Note (Gmail Users):
If you use Gmail, you cannot use your real password. You must enable 2FA on your Google Account and generate an "App Password" to use in the configuration.

SSL/TLS Certificates (Let's Encrypt)

HTTP sends data in plain text (anyone can intercept passwords). HTTPS encrypts it.
Certbot interacts with Let's Encrypt (a free Certificate Authority) to automatically verify you own the domain and install a certificate.

Install Certbot
We use the Nginx plugin (replace with python3-certbot-apache if using Apache).

```
sudo apt install certbot python3-certbot-nginx -y
```

Obtain the Certificate
Prerequisite: Your domain (e.g., mysite.com) must already point to your server's IP address via DNS, or this step will fail.

```
sudo certbot --nginx
```

The Wizard:

1. Enter your email (for renewal warnings).
2. Agree to Terms.
3. Select Domains: Press Enter to select all listed domains.
4. Redirect HTTP to HTTPS? Select 2 (Redirect). This automatically updates your Nginx config to force secure connections.

Auto-Renewal

Let's Encrypt certificates last 90 days. Certbot installs a timer to check this automatically.

1. Verify the Timer:

```
systemctl status certbot.timer
```

2. Test the Renewal Process:

```
sudo certbot renew --dry-run
```

If you see "Congratulations, all renewals succeeded," the automation is working.

Cron Jobs & Scheduling

Cron is the time-based job scheduler. It's the "Alarm Clock" of the server.

Typical uses:

- Running backups at 3:00 AM.
- Deleting old temporary files every Sunday.
- Triggering application schedulers.

Crontab Syntax

To edit the schedule:

```
crontab -e
```

The Syntax:

* * * * * command_to_execute

Position	Value	Description
1	0-59	Minute
2	0-23	Hour (24h format)
3	1-31	Day of Month
4	1-12	Month
5	0-6	Day of Week (0=Sunday)

Examples (Add these to crontab -e)

Database Backup (Daily at 3:30 AM)

```
30 03 * * * mysqldump -u root -p'password' my_db > /home/ubuntu/backups/db.sql
```

Laravel Scheduler (Every Minute)

Laravel has its own internal scheduler. You only need ONE Cron entry to trigger it.

```
* * * * * cd /var/www/html/my_app && php artisan schedule:run >> /dev/null 2>&1
```

Logging Output

By default, Cron is silent. If a job fails, you won't know.

Redirecting Output:

- >> /var/log/myjob.log : Appends standard output to a file.
- 2>&1 : Redirects "Standard Error" to the same place as "Standard Output".

Example:

```
0 5 * * * /path/to/script.sh >> /var/log/script_output.log 2>&1
```

Background Jobs & Queues (Supervisor)

PHP scripts have a timeout (usually 30-60 seconds). If you need to resize a 4K video or send 5,000 emails, you can't do it in the web browser request.

Supervisor is a process manager that ensures this background worker stays alive (restarting it if it crashes).

A. Install Supervisor

```
sudo apt install supervisor -y
```

B. Configure a Worker

Create a configuration file for your specific process.

```
sudo nano /etc/supervisor/conf.d/laravel-worker.conf
```

Paste this config:

```
[program:laravel-worker]
process_name=%(program_name)s_%(process_num)02d
# The command to run the queue listener
command=php /var/www/html/my_app/artisan queue:work --sleep=3 --tries=3 --max-time=3600
# Run automatically?
autostart=true
autorestart=true
# Run as which user? (NEVER root)
user=www-data
# How many copies of the worker to run?
numprocs=2
redirect_stderr=true
stdout_logfile=/var/www/html/my_app/storage/logs/worker.log
```

C. Start the Worker

Whenever you create or edit a config file, you must tell Supervisor to update.

```
# Read the new config file
sudo supervisorctl reread

# Apply the changes
sudo supervisorctl update

# Start/Restart the specific group
sudo supervisorctl start laravel-worker:*

# View Status
sudo supervisorctl status
```

Security & Optimization

SSH Hardening

The default SSH configuration is known to every hacker. Bots constantly scan Port 22 trying to log in as root.

1. Backup the config:

```
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak
```

2. Edit the config:

```
sudo nano /etc/ssh/sshd_config
```

3. Change these lines:
   - Disable Root Login: Find PermitRootLogin and change it to no. (Force users to login as themselves and sudo).
   - Change Port: Find Port 22. Change it to a high, random number, e.g., Port 2222.

4. Update Firewall (CRITICAL STEP): If you restart SSH without opening the new port in the firewall, you will lock yourself out.

```
sudo ufw allow 2222/tcp
```

5. Restart and Test:

```
sudo systemctl restart ssh
```

***Warning**: Do NOT close your current terminal window yet. Open a NEW terminal window and try to connect: ssh -p 2222 user@ip. If it works, you are safe to close the old one.*

Fail2Ban (The Automated Bouncer)

Fail2Ban monitors log files. If it sees too many failed login attempts from a specific IP address within a short time, it updates the firewall to ban that IP temporarily.

1. Install:

```
sudo apt install fail2ban -y
```

2. Configure: Never edit jail.conf. Create a local copy.

```
sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
sudo nano /etc/fail2ban/jail.local
```

3. Enable SSH Protection:

Scroll to the [sshd] section.

```
[sshd]
enabled = true
port    = 2222   # Match your new custom port!
maxretry = 3
bantime = 1h
```

4. Start the Service:

```
sudo systemctl restart fail2ban
```

5. View Banned IPs:

```
sudo fail2ban-client status sshd
```

File & Directory Hardening

Least Privilege: The web server (www-data) should only be able to *read* code, not *write* it. If a hacker finds a vulnerability in your code, write permissions allow them to inject malware permanently.

The "Safe" Structure:

- Directories: 755 (Owner: R/W/X, Others: Read/Execute).
- Files: 644 (Owner: R/W, Others: Read).
- Owner: Your User (for deploying code).
- Group: www-data (for reading code).

Commands:

```
cd /var/www/html

# Reset all directories
sudo find . -type d -exec chmod 755 {} \;

# Reset all files
sudo find . -type f -exec chmod 644 {} \;

# Lock down sensitive config files (like .env or wp-config.php)
sudo chmod 600 .env
```

Caching Overview

Discussion:

- OPCache (Code Cache): PHP is a scripting language. Normally, the server reads the file, compiles it to computer code, and executes it *every time*. OPCache saves the compiled code in RAM so it only happens once.
- Object Cache (Redis/Memcached): Database queries are slow. Object caching saves the *results* of the query in RAM. (e.g., "Give me the list of top 10 posts").

Configuring PHP OPCache

It is usually enabled by default in PHP 8+, but often unoptimized.

1. Edit php.ini: (Replace 8.3 with your version).

```
sudo nano /etc/php/8.3/fpm/php.ini
```

2. Search for [opcache] and unleash it:

```
opcache.enable=1
opcache.memory_consumption=256        # Increase RAM (default is usually small)
opcache.max_accelerated_files=10000
opcache.validate_timestamps=0         # PRODUCTION ONLY!
```

Note: validate_timestamps=0 means PHP stops looking for file changes. The site becomes super fast, but you MUST restart PHP-FPM every time you deploy new code.

Installing Redis (Object Cache)

Redis is generally preferred over Memcached today because it supports data persistence and more complex data types.

1. Install Server and PHP Extension:

```
sudo apt install redis-server php-redis -y
```

2. Verify Redis is running:

```
redis-cli ping
# Output: PONG
```

3. App Configuration: You must tell your app (WordPress/Laravel) to use it.
    - *Laravel (.env):* CACHE_DRIVER=redis
    - *WordPress:* Install a plugin like "Redis Object Cache."

PHP-FPM Tuning

PHP-FPM creates "Workers" (processes) to handle visitors.

- Too few workers: Visitors wait in line (504 Gateway Timeout).
- Too many workers: Server runs out of RAM and crashes.

The Formula: Max Children = (Total RAM - OS Reserved RAM) / Average PHP Process Size

Example: You have 4GB RAM. OS needs 1GB. You have 3GB left. If an average PHP process takes 60MB: 3000MB / 60MB = 50 Workers.

Steps:

1. Edit the Pool Config:

```
sudo nano /etc/php/8.3/fpm/pool.d/www.conf
```

2. Find pm (Process Manager) settings:

```
pm = dynamic                # Good for general use. Use 'static' for high traffic.
pm.max_children = 50        # The calculated limit
pm.start_servers = 10       # Ready on boot
pm.min_spare_servers = 5    # Minimum idle workers
pm.max_spare_servers = 15   # Maximum idle workers
```

3. Restart to Apply:

```
sudo systemctl restart php8.3-fpm
```

Database Optimization Basics

When a website is slow, 80% of the time, the database is the bottleneck. Optimization usually involves two things:

1. Indexing: Organizing the data so the computer doesn't have to read every single row to find one user.
2. Configuration: Allocating enough RAM to the database so it doesn't rely on the slow Hard Drive.

MySQLTuner

Instead of guessing which settings to change in my.cnf, use a script that analyzes your running database and gives you a report.

1. Install & Run:

```
sudo apt install mysqltuner -y
sudo mysqltuner
```

2. Read the Output:

Look for the "Recommendations" section at the bottom.

- *Example:* "Innodb_buffer_pool_size (128M) should be increased to 1G."
- *Action:* You would then edit /etc/mysql/mysql.conf.d/mysqld.cnf and change that specific line.

Creating an Index

Without an index, finding "John Smith" in a phone book requires reading every name from page 1. With an index, you skip straight to "S".

```
-- Log into MySQL
mysql -u root -p

-- Check how slow a query is
SELECT * FROM users WHERE email = 'bob@example.com';
```

```
-- Add an index to the 'email' column
CREATE INDEX idx_email ON users(email);

-- The query is now instant.
```

Monitoring & Maintenance

Log Management (/var/log/)

Linux keeps a diary of everything. When things break, the answer is *always* here.

Log File	What's Inside?
/var/log/syslog	General System. The "Catch-all" for most generic errors.
/var/log/auth.log	Security. SSH logins, sudo usage, and hacking attempts.
/var/log/nginx/error.log	Web Server. PHP crashes and 500 errors.
/var/log/mysql/error.log	Database. Startup failures and crash reports.

Command to Watch Live:
Use tail -f to watch the log update in real-time while you try to reproduce a bug.

```
sudo tail -f /var/log/nginx/error.log
```

Resource Monitoring

htop (CPU & RAM)
The visual task manager.

- Load Average: The 3 numbers at the top (1 min, 5 min, 15 min avg). If these numbers are higher than your CPU core count, the server is overloaded.

Installation

```
sudo apt update
sudo apt install htop
```

To run it, simply type:

```
htop
```

iotop (Disk Usage)
If the server feels slow but CPU is low, the disk is likely the bottleneck.

```
sudo apt install iotop -y
sudo iotop
```

free -m (Memory)
Displays RAM usage in Megabytes.

Automating Security Updates

You don't want to log in every day just to install patches.

1. Install:

```
sudo apt install unattended-upgrades -y
```

2. Configure:

```
sudo dpkg-reconfigure -plow unattended-upgrades
```

3. Select "Yes":

The system will now automatically check for and install Security updates (critical patches) every day, but will usually ignore "Feature" updates (which might break your app).

Backups

The 3-2-1 Rule: 3 copies of data, 2 different media types, 1 off-site.

Database Dumps (mysqldump)

This creates a text file containing all the SQL commands needed to recreate your data.

```
# Basic Backup
mysqldump -u root -p my_app_db > backup.sql

# Professional Backup (With Date Stamp)
mysqldump -u root -p my_app_db > db_backup_$(date +%F).sql
```

Result: You get a file named db_backup_2023-10-27.sql.

File System Backups

A. Archiving (tar)

Compresses a folder into a single file

```
# c=create, z=gzip (compress), v=verbose, f=file
tar -cvf project_backup.tar /var/www/html/my_app
```

Feature	Gzip (.tar.gz)	Bzip2 (.tar.bz2)
Speed	Fast. Compresses and extracts quickly.	Slow. Takes more CPU power and time.
Size	Good compression (Standard).	Better compression (Smaller files).
Usage	Default for most backups and web servers.	Good for long-term storage where size matters most.
Flag	-z	-j

Using Gzip (Standard/Fast) Use the flags -czvf.

```
tar -czvf backup.tar.gz my_project/
```

Using Bzip2 (Smallest Size) Use the flags -cjvf.

```
tar -cjvf backup.tar.bz2 my_project/
```

Extracting Archives (Decompressing)

In modern Ubuntu, you don't actually need to tell tar which compression was used when extracting. It detects it automatically. You just need -xvf.

```
tar -xvf backup.tar.gz
```

Extract a Bzip2 file

```
tar -xvf backup.tar.bz2
```

Extract to a Specific Folder By default, tar extracts to the current folder.
To put files somewhere else, use -C.

```
tar -xvf backup.tar.gz -C /var/www/html/
```

Listing Contents (Without Extracting)
Sometimes you want to see what is inside the "box" without opening it.

```
# -t means "list" (Table of contents)
tar -tvf backup.tar.gz
```

Mirroring (rsync)

Better for huge folders. It checks what has changed and only copies the differences.

```
# Sync local folder to a backup folder
rsync -av /var/www/html/ /backup/website_copy/
```

Remote Backup Strategies

A backup on the same hard drive as the server is useless if the hard drive fails. You must move it Off-Site.

Simple Strategy (SCP to another server):
If you have a second server (or a NAS/Synology at the office).

```
scp db_backup_$(date +%F).sql user@backup-server.com:/home/user/backups/
```

Professional Strategy (Cloud Storage - AWS S3 / Google Drive):
Install a tool called rclone. It allows you to treat Cloud Storage like a local folder.

```
# Example command (after configuring rclone)
rclone copy /home/ubuntu/backups/ remote:bucket-name
```